

Porting Informath to a New Language

GF-RGL and Claude: a Czech case study

Aarne Ranta

July 2026

Outline

What Informath is

GF in five minutes

Porting to Czech

Two thousand words

Results and lessons

What Informath is

The problem

Mathematics exists in two registers.

Formal

- Agda, Rocq, Lean, Dedukti
- machine-checkable
- unreadable to most humans

Informal

- English, French, German, Swedish
- readable
- ambiguous, unverifiable

Informath translates between them — in *all* directions.

Four directions

- formal $\rightarrow$ informal (**informalization**)
- informal $\rightarrow$ formal (**autoformalization**)
- informal $\rightarrow$ informal (translation, *via* the formal)
- formal $\rightarrow$ formal (in special cases)

Two goals:

1. complete informalization of anything expressible in Dedukti;
2. generate *and analyse* informal mathematical language, in all of its variation.

The second goal is inspired by Ganesalingam, *The Language of Mathematics* (2013).

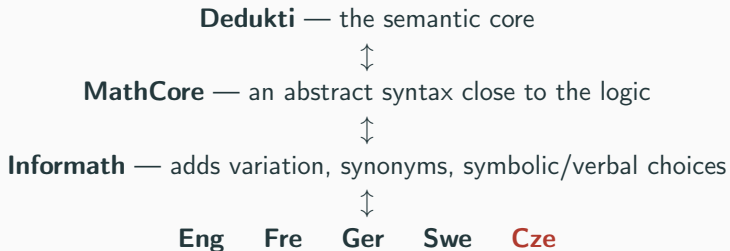
One statement, many languages

```
Dedukti: prop110 : (a : Elem Int) -> (c : Elem Int) ->
  Proof (and (odd a) (odd c)) ->
  Proof (forall Int (b => even (plus (times a b) (times b c))))).
```

```
Rocq:    Axiom prop110 : forall a : Int, forall c : Int,
  (odd a /\ odd c -> forall b : Int, even (a * b + b * c)) .
```

- **En:** Let a and c be integers. Assume that a and c are odd. Then $ab + bc$ is even for all integers b .
- **Fr:** Soient a et c des entiers. Supposons que a et c sont impairs. Alors $ab + bc$ est pair pour tous les entiers b .
- **De:** Seien a und c ganze Zahlen. Nimm an, dass a und c ungerade sind. Dann ist $ab + bc$ gerade für jede ganze Zahl b .
- **Sv:** Låt a och c vara heltal. Anta att a och c är udda. Då är $ab + bc$ jämnt för alla heltal b .

All generated from the Dedukti source. Agda and Lean too.



Agda, Rocq and Lean hang off Dedukti by (partial) conversions.

The natural languages hang off **Informath** by GF concrete syntaxes.

GF in five minutes

Abstract vs. concrete syntax

GF separates *what can be said* from *how it is said*.

```
abstract MathCore = {  
  cat Prop ; Kind ; Ident ;  
  fun CoreAllProp : Kind -> Ident -> Prop -> Prop ;  
}
```

```
concrete MathCoreEng of MathCore = {  
  lincat Prop = S ;  
  lin CoreAllProp kind x prop = ... "for all" ... ;  
}
```

One abstract syntax, n concrete syntaxes $\Rightarrow n^2$ translations, all going through the meaning.

The Resource Grammar Library (RGL)

You do not want to write morphology and word order for every language.

The RGL gives you, per language:

- **Syntax API** — `mkCN`, `mkNP`, `mkS`, `mkVP`, ...
the same names in every language
- **Paradigms API** — `mkN`, `mkA`, `mkV`
language-specific: inflection, gender, cases

40+ languages. Czech is one of them.

Remember that word “one of them”. We will come back to it.

Functors: how porting is *supposed* to work

Write the linearization once, against *interfaces*:

```
incomplete concrete MathCoreFunctor of MathCore =  
  open Syntax, Utilities in { ... }
```

Instantiate per language:

```
concrete MathCoreEng of MathCore = MathCoreFunctor with  
  (Syntax = SyntaxEng), (Utilities = UtilitiesEng) ;  
  
concrete MathCoreCze of MathCore = MathCoreFunctor with  
  (Syntax = SyntaxCze), (Utilities = UtilitiesCze) ;
```

“Functors make it maximally easy to port GF grammars into new languages.” — *Informath under the Hood*

So a new language should be: **write the words, instantiate the functor.**

What a new language needs, in principle

UtilitiesCze	function words, <i>instance</i> of the Utilities interface
CategoriesCze	functor instantiation
MathCoreCze	functor instantiation
MathExtensionsCze	functor instantiation
VerbalConstantsCze	~120 mathematical words
WikidataWordsCze	~2000 mathematical words
InformathCze	glue

Three of these are one-liners. Two are dictionaries.

Nothing here says “rewrite the resource grammar”.

Porting to Czech

The experiment

- Starting point: compilable Czech *templates*, generated by copying Swedish.
- Tool: Claude Code (Opus 4.8), driving the GF compiler in a loop.
- Preceding experiment: the same for Finnish, which went smoothly.
- Question: **how far does the functor story actually carry you?**

Spoiler: the vocabulary was the easy half.

Surprise 0: the templates were Swedish

Not just Swedish *words* — Swedish *morphology*:

```
-- UtilitiesCze.gf, as found
type_CN : CN = mkCN (mkN "typ" "typer") ;
set_N   : N  = mkN "mängd" "mängder" ;
arbitrary_A = mkA "godtycklig" ;
```

- `mkN : Str -> Str -> N` exists in Swedish, **not in Czech**
- `mkPrep : Str -> Prep` in Czech it is `Str -> Case -> Prep`

A “compilable template” compiled only because nothing had been type-checked against the Czech RGL yet.

Surprise 1: ParadigmsCze has no verbs

The whole verb section of the Czech Paradigms API:

```
mkV2 = overload {  
  mkV2 : VerbForms -> VerbForms ** {c : ComplementCase} ;  
  mkV2 : VerbForms -> Case -> ... ;  
  mkV2 : VerbForms -> ComplementCase -> ... ;  
} ;
```

- no mkV
- no mkVS, no mkV3
- no mkPN
- no V3 or Predet *lincat* at all

V3 and Predet silently get GF's default lincat {s : Str}. Nothing warns you.

Building verbs by hand

UtilitiesCze grew its own conjugator for the three productive classes:

```
guessVerbForms : Str -> R.VerbForms = \v ->
  case v of {
    _ + "ovat"  => R.iii_kupovatVerbForms v ;    -- definovat
    st + "ět"   => iVerbForms st v (st + "ěl") ;
    st + "et"   => iVerbForms st v (st + "el") ;
    st + "it"   => iVerbForms st v (st + "il") ;  -- dělit
    st + "at"   => aVerbForms st v ;              -- předpokládat
    -          => aVerbForms v v
  } ;
```

This is a *Paradigms* concern that ended up in an *application* grammar. It is the first sign that the port is not a port.

Surprise 2: the resource grammar is a stub

SentenceCze.gf, in full — four linearizations:

```
lin PredVP, UseCl, UseQCl, UseRC1
```

IdiomCze.gf, in full:

```
concrete IdiomCze of Idiom = CatCze ** open Prelude, ResCze in {  
}
```

Everything else is stubbed in MissingCze as

```
oper ComplVS : VS -> S -> VP = notYet "ComplVS" ;
```

which type-checks, and then explodes at PMCFG generation:

```
gf: Internal error in GeneratePMCFG:  
  evalTerm (Predef.error "NOT YET IMPLEMENTED: ComplVS")
```

Why you cannot work around it

The shared functor reaches these by *qualified* name:

```
-- MathCoreFunctor.gf
VarHypo ident kind = Grammar.ImpP3 (latexNP (mkSymb ident)) (mkVP (useKind kind)) ;
CoreIfProp p q      = Grammar.ExtAdvS ... ;
DefPropJmt ...      = Grammar.SSubjS ... ;
```

- Grammar = GrammarCze. If the line is absent, the name is absent.
- You cannot shadow it from UtilitiesCze.
- You could edit the functor — but it is shared with Eng/Fre/Ger/Swe/Fin.

The only way through the application grammar is to finish the resource grammar.

What had to be added to the Czech RGL

CatCze	lincat VV
SentenceCze	AdvS ExtAdvS SSubjS EmbedS EmbedQS ImpVP
IdiomCze	ImpP3 ExistNP ImpersCl GenericCl
VerbCze	CompCN ComplVS ComplVV PassV2
NounCze	SentCN PredetNP
AdverbCze	SubjS
QuestionCze	QuestIAdv
PhraseCze	UttImpSg UttImpPl UttImpPol
StructuralCze	14 function words: no_Quant every_Det can_VV if_Subj ...
ExtendCze	ExistsNP un-excluded

Found one at a time, each by a compiler crash, each requiring a full RGL rebuild.

The loop, not the linguistics, is what a coding agent is good at.

Three honest approximations

Czech VerbForms has no imperative and no passive participle (passpart is commented out in ResCze).

Rule	Should be	Is
ImpVP	imperative	1pl present: <i>předpokládáme, že ...</i>
PassV2	passive participle	reflexive passive: <i>číslo se dělí</i>
ImpP3	<i>ať</i> (neutral)	<i>necht'</i> (mathematical register)

All three read well *in mathematical Czech* and are wrong in general.

An LLM will happily generate these and not tell you.

They are in the commit message because a human asked “is that actually an imperative?”

Two thousand words

The lexicon: 1177 nouns, 740 adjectives, 86 verbs.

Obvious approach — the one used for Finnish:

```
lin group_Noun = mkNoun "grupa" ;
```

Let GF's smart paradigm guess the declension.
For Finnish this worked. For Czech it is a trap.

The -ce trap

```
-- ResCze.guessNounForms
_ + "ost"          => declKOST s ;
_ + #hardConsonant => declHRAD s ;
_ + "a"           => declZENA s ;
_ + "ce"          => declSOUDCE s ;    -- <<<< masculine ANIMATE
_ + "e"           => declMORE s ;
```

soudce = “judge”. So the guesser declines **matice**, **funkce**, **kružnice** — matrix, function, circle, all feminine — as animate masculine nouns.

Fix: never use the one-argument form. Supply **nominative + genitive + gender**:

```
lin 'group_Noun' = mkNoun (mkN "grupa" "grupy" feminine) ;
lin 'matrix_Noun' = mkNoun (mkN "matice" "matice" feminine) ;
```

The genitive selects the paradigm.

Translating 2003 terms with an LLM

Pipeline

1. extract funs from the abstract syntax
2. attach the English lin, plus the French and German lins as hints
3. split into 8 batches
4. 8 subagents translate in parallel
5. **validate**
6. code-generate the module

The validator is the point

- every fun covered, exactly once
- gender $\in$ 4 legal values
- adjective ends in -ý or -í
- verb ends in -ovat/-at/-it/-et/-ět

Anything else is **rejected**, not repaired.

It caught 4 perfective verbs the grammar cannot conjugate — *uzavřít*, *pokrýt*, *protáhnout*, *použít* — replaced by their imperfectives.

(French and German were only ~25% helpful: those grammars have 537 and 348 of the 2018 terms.)

Then audit what you generated

It compiled. It linked. **It was 19% wrong.**

225 of 1177 nouns fell through declensionNounForms to the soft-declension fallback. Two causes, both in the RGL.

Bug 1: half the consonants were never matched

```
hardConsonant    : pattern Str = #("d"|"t"|"g"|"h"|"k"|"n"|"r") ;  
neutralConsonant : pattern Str = #("b"|"f"|"l"|"m"|"p"|"s"|"v") ;    -- never used!
```

Czech *neutral* consonants take the *hard* endings. But the paradigm selector only tested `hardConsonant`:

graf, atom, profil, přístup, uzel → soft declension

119 nouns. Also present in `guessNounForms`: `mkN "graf"` was *already* wrong for every Czech RGL user.

Bug 2: the genitive was thrown away

`mkN "uzel" "uzlu" mascInanimate` — the genitive is used only to *pick* a paradigm, then discarded. The stem is re-guessed from the nominative by `dropFleetingE`, which guesses wrong *both ways*:

	was	should be
člen (member!)	čl <u>nu</u>	členu
uzel (knot)	uzel <u>u</u>	uzlu
doplňěk (complement)	doplň <u>ěku</u>	doplňku
výpočet (computation)	výpoč <u>etu</u>	výpočtu

Fix: give `declHRAD`, `declHRADA`, `declMUZ` stem-taking variants, and pass the stem the caller already supplied. One-argument `mkN` keeps its old behaviour.

Seven missing paradigms

declLATINUS	algoritmus – algoritmu	35 nouns
declLATINUM	kontinuum – kontinua	14
declGREEKMA	schéma – schématu	5
declHRADA	les – lesa, zákon – zákona	6
declADJF	proměnná – proměnné (<i>variable!</i>)	2
declADJM	nultý – nultého	1
declINVAR	bombé, tamari, software	7

Fallbacks: 225 $\longrightarrow$ 1
(the remaining one is the acronym *ADE*)

Results and lessons

It speaks Czech

```
VarHypo (StrIdent "x") (NounKind group_Noun)
  ==>  necht' $ x $ je grupa

CoreAllProp (NounKind ring_Noun) (StrIdent "R") FalseProp
  ==>  pro všechny okruhy $ R $ , máme spor

CoreExistProp (NounKind matrix_Noun) (StrIdent "M") FalseProp
  ==>  existuje matice $ M $ , takový, že máme spor

CoreIfProp FalseProp FalseProp
  ==>  jestliže máme spor , pak máme spor
```

Spot the bug: **takový** should be *taková* — *matice* is feminine.

`such_that_Subj` is a frozen string, and the Czech `Subj = {s : Str}` `lincat` cannot carry agreement. Still open.

informath		
UtilitiesCze, VerbalConstantsCze rewritten	~250 lines	
WikidataWordsCze generated	2003 lins	

gf-rgl (unplanned)		
missing linearizations, 9 modules	21 rules	
noun paradigm selector + 7 paradigms	~120 lines	

Roughly half the work was in the library that was supposed to already exist.

Two of the RGL bugs affect **every** Czech GF grammar, not just Informath.

What the LLM was good at

- **The compile loop.** Crash → read module → write lin → rebuild RGL → repeat. Twenty times. Tirelessly.
- **Reading an unfamiliar codebase.** Finding that V3 has no lincat, that ExistsNP reduces to ExistNP, that MissingCze stubs explode only at PMCFG time.
- **Bulk lexicography.** 2003 terms, 8 agents, a few minutes.
- **Auditing its own output** — *when told to*. The 225-noun bug was found by writing a script that re-implemented the paradigm selector and diffed it against the generated file.

What it was bad at

- **Knowing when “it compiles” is not “it works”.**
linking ... OK with 19% of nouns mis-declined.
- **Volunteering that an approximation is an approximation.**
ImpVP as a present tense is a reasonable hack — and would have shipped silently.
- **Knowing the register.** *necht'* vs *at'*; whether *dělitel* is grammatically animate. It guessed right often, and confidently either way.
- **Rare vocabulary.** ~1500 of the 2003 terms have no French/German gloss and no established Czech word. Those are coinages. **They have had no native review.**

Lessons for the next language

1. Check the RGL first.

`wc -l src/<lang>/SentenceCze.gf` tells you more about the cost than the size of the lexicon does. “Language X is in the RGL” is not a binary fact.

2. Do not trust smart paradigms in a rich morphology.

Supply nominative, genitive and gender. The extra column is cheap; the guesser’s mistakes are invisible.

3. Put a validator between the LLM and the file.

Reject, do not repair. It caught 4 unconjugable verbs immediately, and made “complete” a checkable claim rather than a hope.

4. Audit the generated artefact against the library, not against itself.

Re-implement the selector in 30 lines of Python, and diff.

5. Fix the library.

The paradigm bugs were 20 years old and would have been papered over by a per-word workaround.

- `such_that_Subj` does not agree in gender — needs an agreeing Subj lincat in Czech.
- `map_V3`, `range_V3` linearize to nothing (no V3 in Czech).
- Multiword nouns inflect only their head:
vlastní čísla, not *vlastního čísla*.
- `ResCze.VerbForms` needs an imperative and a passive participle; then `ImpVP`, `PassV2`, `ImpP3` should be revisited.
- **Native review of 2003 terms.**
- `UserExtensionsCze` still disabled.

The functor really does make the *grammar* port easy.

What it cannot do is make the *resource grammar* exist.

An LLM turns a months-long lexicography-and-debugging task into a day — provided every claim it makes is mechanically checkable, and provided somebody keeps asking “*is that actually an imperative?*”

github.com/GrammaticalFramework/informath